DDA Process Analyst Thinking Process

Executive Summary

The DDA (Discover-Design-Architecture) methodology is the intellectual framework behind BitX's 26-agent SDLC platform. This document examines how the Process Analyst (PA) agent — a key Discover-phase agent — applies the DDA methodology to systematically analyze business processes, identify bottlenecks, and produce actionable process documentation.

The PA agent demonstrates the engine's thinking process through its 6-stage pipeline execution: understanding the request, gathering contextual knowledge, planning an analysis approach, executing with tools, validating outputs, and delivering structured artifacts. This document provides a deep dive into the PA's decision trees, skills, artifact formats, and cross-agent collaboration patterns.

---

1. DDA Methodology Overview

1.1 What is DDA

DDA stands for **Discover-Design-Architecture** — a three-phase approach to software development that mandates understanding before designing, and designing before building:

```
+------------------+     +------------------+     +------------------+
|     DISCOVER     |     |      DESIGN      |     |   ARCHITECTURE   |
|                  |     |                  |     |    (Build/Test/   |
|  Understand the  |---->|  Design the      |---->|     Deploy)      |
|  problem space   |     |  solution space  |     |  Construct the   |
|                  |     |                  |     |  solution        |
+------------------+     +------------------+     +------------------+
       Gate 1                   Gate 2                   Gate 3
```

1.2 DDA Applied to 26 Agents

Phase	Agents	Focus
Discover (3)	Product Manager, Business Analyst, Process Analyst	Requirements, processes, stakeholders
Design (4)	Solutions Architect, UX Designer, System Analyst, Technical Writer	Architecture, UI/UX, system design, documentation
Build (6)	Backend Dev, Frontend Dev, Full Stack Dev, DB Admin, Data Engineer, DevOps	Code, database, infrastructure
Test (10)	QA Lead + 9 specialized testers	Quality assurance across all dimensions
Deploy (3)	Release Manager, Security Analyst, Technical Writer	Release, security, documentation

1.3 Phase Gates

Phase transitions require quality gates:

Gate	Transition	Criteria
Gate 1	Discover to Design	Requirements documented, user stories with acceptance criteria, process flows validated
Gate 2	Design to Build	ADRs approved, API specs complete, database schemas reviewed
Gate 3	Build to Deploy	80% test coverage, all tests pass, security scan pass, UAT sign-off

The Process Analyst is a **Gate 1 contributor** — its outputs (process flows, stakeholder maps, gap analyses) are prerequisites for transitioning from Discover to Design.

---

2. Process Analyst Agent Profile

2.1 Identity

Property	Value
ID	`process_analyst`
Name	Process Analyst
Prefix	PA-
Phase	Discover
Model Role	Router (Ministral 8B)
Approval Required	No

2.2 Core Responsibilities

1. **Map current-state business processes** with all stakeholders
2. **Create BPMN process flow diagrams** in Mermaid syntax
3. **Identify process gaps**, pain points, and optimization opportunities
4. **Document stakeholder roles** using RACI matrices
5. **Define business rules and data flows** for each process

2.3 Quality Criteria

The PA agent is evaluated against 5 quality standards:

Criterion	Metric
Every process has identified **trigger, owner, and outcome**	Completeness of process documentation
All decision points have **at least 2 branches** documented	Branch coverage in flow diagrams
Pain points are ranked by **impact (HIGH/MEDIUM/LOW)**	Gap analysis prioritization
Mermaid diagrams **render correctly**	Technical validity of output
Stakeholder RACI matrix **covers all roles**	Completeness of stakeholder mapping

2.4 Boundary Rules

Always:

- Include swimlane diagrams showing actor responsibilities
- Document business rules with validation and exception handling
- Map data flows between systems
- Reference existing Product Hub entities (Product, Epic, Feature, Story, Task)

Ask First:

- Recommending process elimination or major restructuring
- Adding new stakeholder roles not in original scope

Never:

- Skip current-state analysis and jump to recommendations
- Make technology choices (delegate to Solutions Architect)
- Write code or database queries

---

3. Thinking Process: Stage-by-Stage

The PA agent's "thinking" is driven by the Orchestrator's 6-stage pipeline. Here is how each stage manifests for process analysis tasks.

3.1 Stage 1: Intake — Understanding the Request

What happens:

1. The Orchestrator builds a system prompt from the PA profile (role, responsibilities, quality criteria, boundaries, tool usage instructions)
2. The user's request is sent alongside the system prompt to the Router model
3. The Router model parses the request and produces an initial understanding

PA-specific behavior:

- Identifies the **process name** or domain area
- Determines the **scope** (current-state analysis, gap analysis, or stakeholder mapping)
- Notes any **stakeholders** or systems mentioned
- Classifies the task against its 5 skills

Example intake output:

```
Understanding: The user wants an analysis of the tenant onboarding process.
This is a current-state analysis (skill: /pa process).
Scope: tenant lifecycle from creation to activation.
Key stakeholders: Admin, System, Tenant User.
Related Product Hub entities: Tenant, Product.
```

3.2 Stage 2: Context Gathering — Research Phase

What happens:

1. **RAG search**: The engine queries the knowledge base with the user's prompt, retrieving the top 5 most relevant documents (BM25 score >= 0.05)
2. **Injected context**: RAG results are formatted and injected into the conversation
3. **Model-directed exploration**: The Router model decides what additional files or docs to read (up to 5 tool calls)

PA-specific behavior:

- RAG retrieves existing process documentation, schema files, and related agent outputs
- PA typically calls `search_docs` to find related process flows
- PA calls `read_file` on relevant documentation files
- PA may call `search_content` to find Product Hub entity references

Context injection format:

```
## Relevant Context (RAG)

### [1] tenant-management (doc, score: 0.892)
Tenant management allows administrators to create and manage organizational tenants...

### [2] process-analyst (profile, score: 0.654)
Analyzes business processes, creates workflow diagrams...
```

3.3 Stage 3: Planning — Structured Approach

What happens:

1. The Router model creates a step-by-step plan based on the enriched context
2. The plan is saved as a `plan` artifact

Typical PA plan structure:

```
Step 1: Document the current-state tenant onboarding process
  - Identify trigger event (admin creates tenant)
  - Map all actors (Admin, System, New Tenant User)
  - Document each step in the process

Step 2: Create BPMN flow diagram
  - Design swimlane diagram with actor lanes
  - Add decision points (validation, approval)
  - Include exception flows

Step 3: Identify stakeholders and create RACI matrix
  - List all roles involved
  - Assign Responsible/Accountable/Consulted/Informed

Step 4: Identify gaps and pain points
  - Compare to industry best practices
  - Rank issues by impact

Step 5: Write summary with recommendations
```

3.4 Stage 4: Tool Loop — Execution

What happens: The Worker (or Router, since PA uses router) executes the plan using tools. Maximum 15 iterations with up to 3 tool calls each.

PA tool usage pattern:

Tool	When Used	Purpose
`search_files`	Early	Find existing process documentation files
`read_file`	Early-Mid	Read existing docs, schemas, entity definitions
`search_content`	Mid	Find references to processes, entities, stakeholders
`search_docs`	Early	Query documentation for related processes
`write_file`	Late	Write process flow documents, BPMN diagrams, RACI matrices

Iteration pattern:

1. Iterations 1-3: Research (reading existing docs and code)
2. Iterations 4-6: Analysis (searching for patterns and stakeholders)
3. Iterations 7-10: Writing (creating artifacts)
4. Signal `<done>` when all deliverables are complete

3.5 Stage 5: Validation

Validators applied to PA runs:

Validator	Expected Result
`blocked_file_patterns`	PASS — PA should not touch sensitive files
`tool_restrictions`	PASS — PA is router role; write tools are checked but PA is not forbidden from them in its profile
`output_schema`	PASS — Artifacts should have valid structure
`approval_required`	SKIP — PA does not require approval

3.6 Stage 6: Finalize

The Router model summarizes all work performed:

- Lists all artifacts created
- Highlights key findings
- Notes any open questions or recommendations
- Aggregates token usage and duration

---

4. Decision Trees

4.1 Request Classification

```
Request Received
    |
    +-- Is it about a process? ---------> Yes --> What type?
    |                                              |
    |                                   +---------+---------+
    |                                   |         |         |
    |                              Current    Gap       Flow
    |                              State    Analysis   Diagram
    |                                |         |         |
    |                             /pa process /pa gap   /pa bpmn
    |                                        -analysis  /pa flow
    |
    +-- Is it about stakeholders? ---> Yes --> /pa stakeholder-map
    |
    +-- Is it about data flows? -----> Yes --> /pa process (with data focus)
    |
    +-- Is it about business rules? -> Yes --> /pa process (with rules focus)
    |
    +-- Unclear ----------------------> Ask for clarification
```

4.2 Complexity Assessment

Complexity	Criteria	Expected Iterations
Simple	Single actor, linear flow, < 5 steps	3-5 tool loop iterations
Medium	2-3 actors, decision points, 5-15 steps	6-10 iterations
Complex	4+ actors, parallel paths, exception handling, > 15 steps	10-15 iterations

4.3 Tool Selection Logic

```
Need information about existing docs?
    --> search_docs (keyword search across .md, .yaml, .txt)

Need to read a specific file?
    --> read_file (with path from previous search)

Need to find code references?
    --> search_content (regex across source files)

Need to write a deliverable?
    --> write_file (process flow, BPMN, RACI, gap analysis)

Need to find file locations?
    --> search_files (glob pattern matching)
```

---

5. Skills Deep Dive

5.1 `/pa process` — Current-State Analysis

Input: Process name (e.g., "tenant onboarding")

Thinking steps:

1. Identify the **trigger** — what initiates the process
2. Map all **actors** — who is involved
3. Document each **step** in sequence
4. Note **decision points** with all branches
5. Document **exceptions** and error flows
6. Identify the **outcome** — what marks completion

Output structure:

```
# Current-State Process: [Process Name]

## Overview
- Trigger: [what starts it]
- Owner: [who is responsible]
- Outcome: [what indicates completion]

## Actors
- [Actor 1]: [role description]
- [Actor 2]: [role description]

## Process Steps
1. [Step description]
   - Actor: [who performs it]
   - Input: [what is needed]
   - Output: [what is produced]
   - Business Rules: [validation/constraints]

## Decision Points
- [Decision]: [Branch A] | [Branch B]

## Exceptions
- [Exception scenario]: [handling]
```

5.2 `/pa bpmn` — BPMN Flow Diagram

Input: Process name

Thinking steps:

1. Identify start and end events
2. Map activities (tasks performed by actors)
3. Add gateways (decision points, parallel splits)
4. Create swimlanes per actor
5. Render in Mermaid syntax

Output: Mermaid flowchart with swimlanes

```
graph LR
    subgraph Admin
        A[Create Tenant] --> B{Validate?}
        B -->|Yes| C[Save Tenant]
        B -->|No| D[Show Errors]
    end
    subgraph System
        C --> E[Send Welcome Email]
        E --> F[Activate Tenant]
    end
```

5.3 `/pa flow` — User Flow Diagram

Input: Scenario name (e.g., "first-time signup")

Output: Mermaid sequence or flowchart showing user interactions

5.4 `/pa gap-analysis` — Gap Identification

Input: Implicit (requires existing current-state analysis)

Thinking steps:

1. Review the current-state documentation
2. Compare against target state or best practices
3. Identify deltas (missing steps, missing actors, missing rules)
4. Rank each gap by impact

Output structure:

```
# Gap Analysis: [Process Name]

## Gaps Identified

### HIGH Impact
- [Gap]: [description] — [recommendation]

### MEDIUM Impact
- [Gap]: [description] — [recommendation]

### LOW Impact
- [Gap]: [description] — [recommendation]

## Summary
```

```
- Total gaps: [N]
- High: [N], Medium: [N], Low: [N]
```

5.5 `/pa stakeholder-map` — RACI Matrix

Input: Feature name

Thinking steps:

1. Identify all roles involved
2. For each process activity, assign R/A/C/I
3. Validate that every activity has exactly one Accountable

Output: RACI matrix table

Activity	Admin	Developer	QA	PM	User
Define Requirements	C	I	I	R/A	C
Build Feature	I	R	C	A	I
Test Feature	I	C	R	A	I
Deploy	A	R	C	I	I

R = Responsible, A = Accountable, C = Consulted, I = Informed

---

6. Artifact Specifications

Artifact	Type	Format	Purpose
`PROCESS_FLOW`	Markdown + Mermaid	Structured document with embedded diagrams	Complete process documentation
`CURRENT_STATE_ANALYSIS`	Markdown	Structured analysis with sections	Baseline understanding of how things work today
`GAP_ANALYSIS`	Markdown	Prioritized list with impact ratings	Identifies what needs to change
`STAKEHOLDER_MAP`	Markdown table	RACI matrix	Clarifies who does what

All artifacts are stored in the Run Store's `agent_artifacts` table with the run ID, step ID, and content.

---

7. Bottleneck Identification Methodology

7.1 Pattern Recognition

The PA agent identifies bottlenecks by looking for these patterns:

Pattern	Indicator	Example
Manual handoffs	Automated step followed by manual step	System generates report, admin manually emails it
Missing decision criteria	Gateway without documented conditions	"Approved?" with no approval criteria defined
Single point of failure	One actor handles critical path alone	Only admin can approve tenants
Data re-entry	Same data entered in multiple systems	User info in auth system AND CRM

Missing exception handling	Happy path documented, errors not	What happens if email fails?

7.2 Impact Classification

Level	Definition	Criteria
HIGH	Blocks multiple downstream processes	Causes cascading delays; affects Gate 1 passage
MEDIUM	Causes delays but has workarounds	Process continues but inefficiently
LOW	Minor friction, optimization opportunity	Annoyance but not blocking

7.3 Common Bottleneck Categories

1. **Process bottlenecks**: Steps that create queues or waiting
2. **Information bottlenecks**: Missing data or unclear handoff content
3. **Decision bottlenecks**: Unclear authority or missing criteria
4. **Technology bottlenecks**: Manual steps that could be automated
5. **Organizational bottlenecks**: Wrong person assigned to a task

---

8. Cross-Agent Collaboration

8.1 Upstream: What PA Receives

Source	What PA Gets
Product Manager	Product vision, feature descriptions, priority context
Existing documentation	Current process docs, system descriptions

8.2 Downstream: What PA Produces

Consumer	What PA Delivers
Solutions Architect	Process flow diagrams (for system design decisions)
System Analyst	Process flows (for requirements derivation)
Product Manager	Stakeholder maps (for planning and resource allocation)
Business Analyst	Gap analysis (for user story creation)
Backend Developer	Business rules (for implementation logic)
Database Admin	Business rules (for data model constraints)

8.3 Handoff Quality

A good handoff from PA to downstream agents includes:

- **Clear naming**: Artifacts named by process (e.g., "Tenant Onboarding Process Flow")
- **Mermaid diagrams**: Machine-renderable, not just text descriptions
- **Prioritized gaps**: Downstream agents know what to address first
- **Business rules**: Explicit validation rules, not assumptions
- **Actor identification**: Who is responsible at each step

---

9. Worked Example

9.1 Request

> *"Analyze the user registration process for the Product Hub platform"*

9.2 Intake Understanding

```
Process: User Registration
Domain: Product Hub authentication
Scope: Full registration flow from landing page to activated account
Expected artifacts: Current-state analysis, BPMN diagram, RACI matrix
```

9.3 Context Gathered

RAG returns:

- Product Hub authentication documentation (score: 0.85)
- User entity schema (score: 0.72)
- Existing auth flow description (score: 0.68)

Tool calls:

- `search_docs("user registration")` — finds 3 related docs
- `read_file("auth/README.md")` — reads auth module documentation
- `search_content("register")` — finds registration controller code

9.4 Plan Created

```
1. Document current-state registration process (trigger, actors, steps)
2. Create BPMN swimlane diagram (User, Frontend, Backend, Database, Email)
3. Create RACI stakeholder matrix
4. Identify gaps and pain points
5. Write summary with recommendations
```

9.5 Tool Loop Execution

Iteration	Tools Used	Action
1	`read_file`	Read auth controller for registration logic
2	`search_content`	Find email verification flow
3	`read_file`	Read user entity/schema
4	`write_file`	Write current-state analysis document
5	`write_file`	Write BPMN Mermaid diagram
6	`write_file`	Write RACI matrix
7	`write_file`	Write gap analysis
8	Signal `<done>`	All artifacts complete

9.6 Final Artifacts

1. **CURRENT_STATE_ANALYSIS**:
    - Trigger: User clicks "Register" button
    - Actors: User, Frontend App, Backend API, Database, Email Service

    - 8 steps from form submission to account activation
    - 3 decision points (validation, email verification, duplicate check)

2. **PROCESS_FLOW** (Mermaid BPMN):
    - 5 swimlanes (one per actor)
    - 8 activities, 3 gateways, 2 exception flows
3. **STAKEHOLDER_MAP** (RACI):
    - 5 activities mapped across Admin, Developer, QA, PM roles
4. **GAP_ANALYSIS**:
    - HIGH: No email bounce handling
    - HIGH: No rate limiting on registration endpoint
    - MEDIUM: No social login option
    - LOW: No progress indicator during registration

Appendix: Mermaid Diagram Templates

BPMN-Style Process Flow

```
graph LR
    subgraph Actor1["Admin"]
        A1[Task 1] --> A2{Decision?}
    end
    subgraph Actor2["System"]
        A2 -->|Yes| B1[Task 2]
        A2 -->|No| B2[Error Handler]
        B1 --> B3[Task 3]
    end
```

Sequence Diagram

```
sequenceDiagram
    participant U as User
    participant F as Frontend
    participant B as Backend
    U->>F: Submit Form
    F->>B: POST /api/register
    B-->>F: 201 Created
    F-->>U: Success Message
```

RACI Template

```
| Activity         | Role A | Role B | Role C |
|----------------|--------|--------|--------|
| [Activity 1]    | R      | A      | I      |
| [Activity 2]    | C      | R/A    | I      |
```

Cross-references: [AI Engine Architecture](#) (Section 2: Orchestrator Pipeline) | [Learning Methodology](#) (Section 5: Context Gathering)